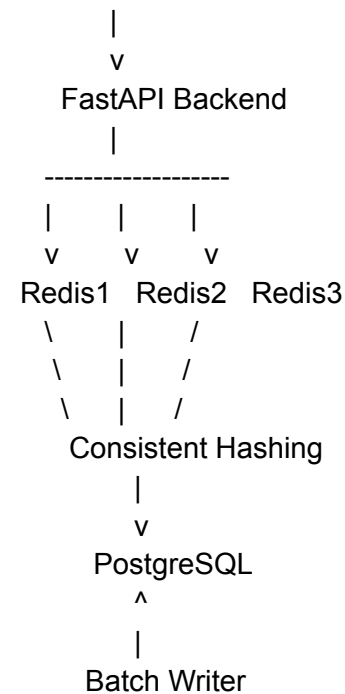


Search Typeahead System – Project Report

1. Architecture Diagram and Explanation

Frontend (HTML/CSS/JavaScript)



Architecture Explanation

The system follows a distributed architecture consisting of four major components:

1. **Frontend**
 - Provides a search box with real-time suggestions.
 - Displays trending searches and performance metrics.
2. **FastAPI Backend**
 - Handles all API requests.

- Routes requests to cache or database.
- Manages batch writes and trending logic.
- 3. **Redis Distributed Cache**
 - Three Redis nodes are used.
 - Consistent hashing determines which node stores a prefix.
 - Frequently accessed suggestions are served directly from Redis.
- 4. **PostgreSQL**
 - Stores all search queries and counts.
 - Acts as the primary persistent datastore.
- 5. **Batch Writer**
 - Aggregates search updates in memory.
 - Flushes updates periodically to reduce database writes.

2. Dataset Source and Loading Instructions

Dataset

The dataset contains search queries and popularity counts.

Dataset size:

- Total records: **100,001**
- Format:

```
query,count  
iphone,58615  
spotify,95050  
github,82344
```

Loading Instructions

1. Start PostgreSQL container:

```
docker compose up -d
```

2. Run the loader:

```
python load_data.py
```

3. Verify data:

SELECT COUNT(*) FROM queries;

Expected result:

100001

3. API Documentation

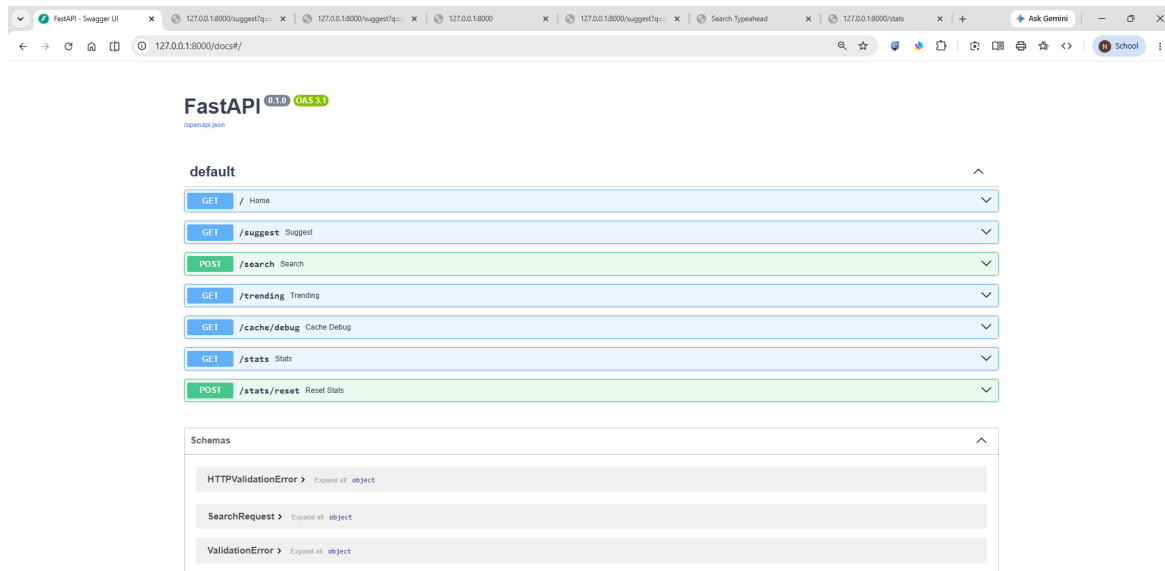


Figure 1: FastAPI Swagger documentation showing all available endpoints.

GET /suggest

Fetch typeahead suggestions.

Example:

GET /suggest?q=iphone

Response:

```
{
  "results": [
    {
      "query": "iphone 15",
      "count": 96927
    }
  ]
}
```

```
}  
],  
  "latency_ms": 1.02,  
  "source": "redis"  
}
```

POST /search

Submit a search query.

Request:

```
{  
  "query": "github"  
}
```

Response:

```
{  
  "message": "Searched"  
}
```

GET /trending

Returns top trending searches.

Example:

GET /trending

GET /cache/debug

Shows cache routing information.

Example:

GET /cache/debug?prefix=iphone

Response:

```
{
  "node": "node2",
  "hit": true
}
```

GET /stats

Returns system metrics.

Response:

```
{
  "cache_hits": 5,
  "cache_misses": 2,
  "hit_rate_percent": 71.4
}
```

4. Design Choices and Trade-offs

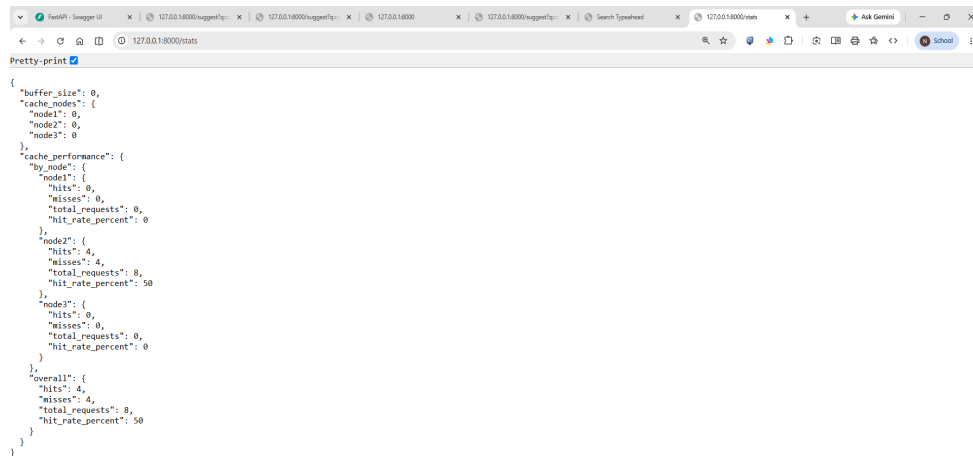


Figure 2: Cache distribution and performance statistics across Redis nodes.

Why PostgreSQL?

Advantages:

- Reliable persistence
- Strong query support

- Easy local deployment

Trade-off:

- Slower than cache access

Why Redis?

Advantages:

- Extremely fast reads
- Reduces database load

Trade-off:

- Additional infrastructure complexity

Why Consistent Hashing?

Advantages:

- Even distribution across cache nodes
- Scalable architecture

Trade-off:

- Slightly more complex implementation than a single cache

Why Batch Writes?

Advantages:

- Reduces database write pressure
- Improves throughput

Example:

Without batching:

100 searches → 100 database writes

With batching:

100 searches → 1 database write

Trade-off:

If the application crashes before flush, some updates may be lost.

Trending Search Design

Ranking formula:

$\text{score} = \text{count} + \text{recent_score}$

Recent activity:

$\text{recent_score} += \text{count} * 10$

Decay mechanism:

$\text{recent_score} *= 0.95$

This prevents temporary spikes from permanently dominating rankings.

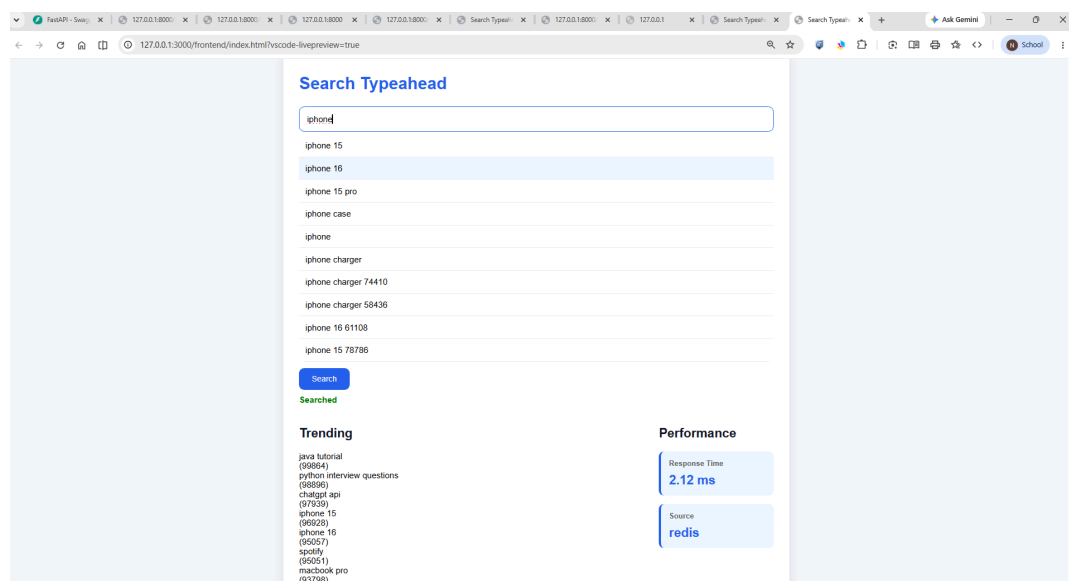
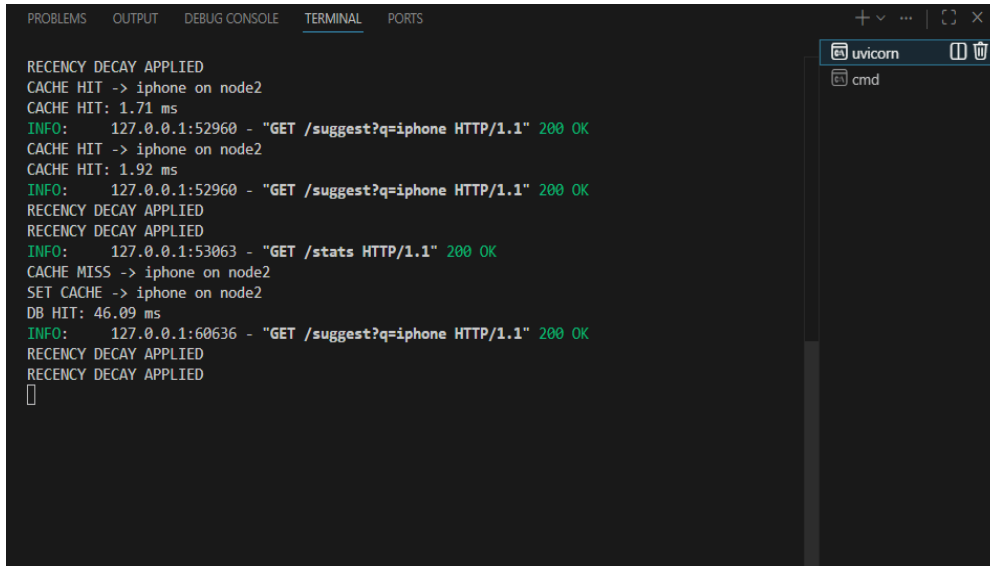


Figure 3: Frontend dashboard displaying suggestions, trending searches, response time, and data source.

5. Performance Report



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
REGENCY DECAY APPLIED
CACHE HIT -> iphone on node2
CACHE HIT: 1.71 ms
INFO: 127.0.0.1:52960 - "GET /suggest?q=iphone HTTP/1.1" 200 OK
CACHE HIT -> iphone on node2
CACHE HIT: 1.92 ms
INFO: 127.0.0.1:52960 - "GET /suggest?q=iphone HTTP/1.1" 200 OK
REGENCY DECAY APPLIED
REGENCY DECAY APPLIED
INFO: 127.0.0.1:53063 - "GET /stats HTTP/1.1" 200 OK
CACHE MISS -> iphone on node2
SET CACHE -> iphone on node2
DB HIT: 46.09 ms
INFO: 127.0.0.1:60636 - "GET /suggest?q=iphone HTTP/1.1" 200 OK
REGENCY DECAY APPLIED
REGENCY DECAY APPLIED
[]
```

Figure 4: Backend logs showing cache hits, cache misses, and recency decay execution.

Latency Measurements

Operation	Average Latency
PostgreSQL Query	30–80 ms
Redis Cache Hit	1–2 ms

Performance Improvement

Redis cache provides approximately:

30x – 80x faster responses

compared to direct database access.

Cache Metrics

The system tracks:

- Cache Hits
- Cache Misses
- Hit Rate %
- Per-node cache statistics

Distributed Cache Verification

Three Redis nodes are used:

- Node 1
- Node 2
- Node 3

Consistent hashing distributes prefixes across nodes.

Batch Write Verification

Observed behavior:

FLUSHING {'spotify': 20}

This demonstrates aggregation of repeated searches before writing to PostgreSQL.

Conclusion

The project successfully implements a scalable search typeahead system with:

- FastAPI backend
- PostgreSQL datastore
- Distributed Redis cache
- Consistent hashing
- Batch writes
- Trending search ranking
- Real-time performance monitoring

The architecture improves latency significantly while reducing database load and maintaining scalable system behavior.